

1조 1차 발표 정리

데이터 전처리

군집 분석

추천 알고리즘

구현

결과 정리

1조

프로젝트 결과 발표

팀원 : 이진규, 이한웅, 최혜정

국민대학교, AI 응용시스템의 이해와 구축

Table of Contents

01	02	03	04	05
- 1차 발표 리뷰 1차 발표 내용을 간단 리뷰하고, 프로젝트 목표를 다시 정리합니다.	- 데이터 전처리 필요에 맞게 데이터를 선정하고, 데이터를 전처리한 과정을 설명합니다.	✦ 군집 분석 어떤 목적으로 군집을 분류한 것인지 설명하고, 군집 결과를 평가합니다.	✦ 추천 알고리즘 추천 알고리즘의 적용을 설명합니다.	- 간단 구현 및 프로젝트 결과 추천 시스템 적용을 간단하게 구현해 보고 프로젝트 결과를 리뷰합니다.



Team Project " 롯데멤버스 고객 구매데이터를 기반으로 한 추천 알고리즘 적용 "



프로젝트 목표

①

데이터 탐색을 통한 인사이트 도출(1차 발표)

→ 구매 데이터 비중이 높은 A, B 사 고려, 그중 B사의 구매내용은 주로 생필품이므로 상품추천을 위해 A사로 타겟팅.

②

군집분석 적용 계획

→ 고객을 분류하여 추천 알고리즘을 적용함으로써, 계산의 복잡도를 줄이고 성능을 개선하고자 함.

③

추천 알고리즘 적용 계획

→ 신규 고객 유입 및 기존 고객의 편의성과 만족도를 높이는 추천 시스템 적용.



데이터 선정 및 전처리

- ☑ 데이터 탐색을 통한 A 제휴사 타겟팅 결정, 고객 구매데이터 선정.
- ☑ Recency, Frequency, Monetary
→ 최근 구매시기, 구매빈도, 구매금액 파생변수 생성
- ☑ RFM score 계산하여 고객 가치 점수화

RFM SCORE ?

→ 고객의 마지막 쇼핑, 방문 빈도, 쇼핑 금액 등을 취합하여 누적 계산

$$(Recency\ score \times Recency\ weight) + (Frequency\ score \times Frequency\ weight) + (Monetary\ score \times Monetary\ weight)$$

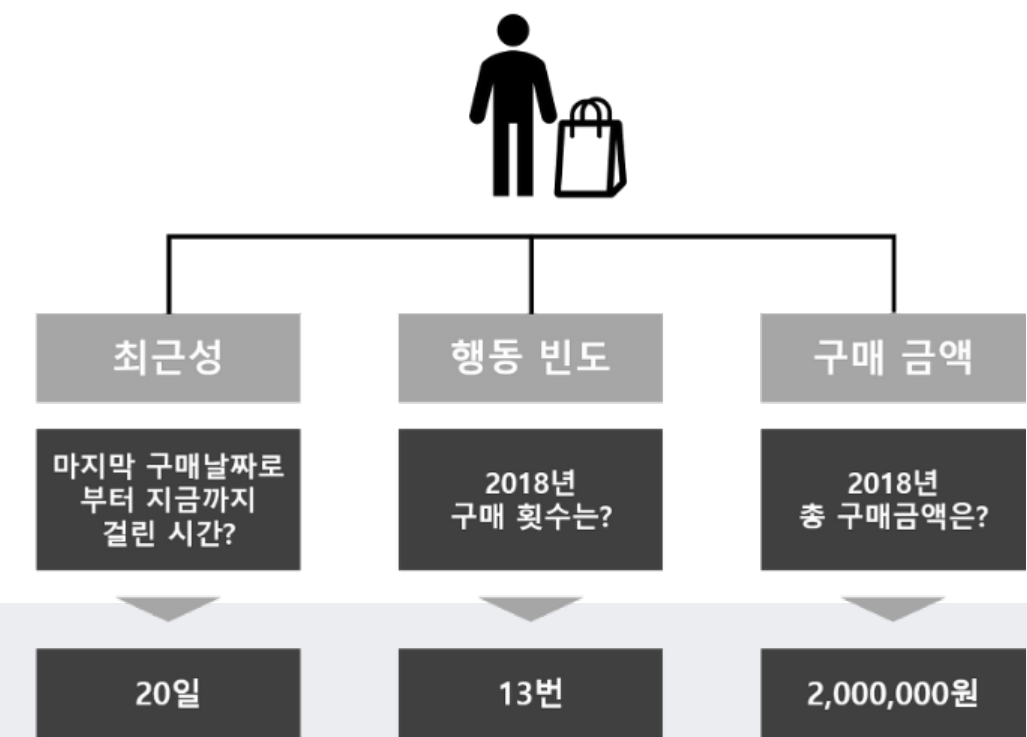
데이터셋

cno	gender	age_grp	scls_c_nm	R	F	M
1	M	60세이상	Bag&Bag	135	1	186200
1	M	60세이상	L.B	277	1	85500
1	M	60세이상	L/C 아웃도어	186	1	90250
1	M	60세이상	N.B	227	1	251750
1	M	60세이상	global SPA	310	2	54800
:	:	:	:	:	:	:
19383	F	25세~29세	수입브릿지	30	1	149000
19383	F	25세~29세	아디다스의류	78	1	24000
19383	F	25세~29세	영스트리트	81	2	51900
19383	F	25세~29세	컬처캐주얼	79	1	62100

- 구매내역 TR 테이블에서 'A'(백화점 추정)만 가져옴.
→ `tr1 = tr[tr['제휴사'] == 'A']`
- 고객Demo, 상품분류체계 맵핑
→ `pd.merge(tr1, demo, on='고객번호', how='left')`

Recency, Frequency, Monetary

- Recency : 가장 최근 구매 날짜 또는 가장 최근 구매 날짜 이후의 시간 간격.
- Frequency : 총 구매 수. 이 정보는 빈도 점수를 계산하는 데 사용.
- Monetary : 모든 구매의 구매총액 집계값.



① 고객별 소분류명별 최근 구매일수 구하기 (2015년 12월 01일과 최근 구매일자와의 차이)

고객별, 소분류별, 날짜별로 정렬

```
filter_period = filter_period.sort_values(by=['cno','gender', 'age_grp','scls_c_nm', 'de_dt'], ascending=[True, True, True,True,False])
```

고객별, 소분류별 최근 구매일자 추출

```
recent_purchase_days = filter_period.groupby(['cno','gender', 'age_grp','scls_c_nm'])['de_dt'].first().reset_index()
```

```
recent_purchase_days['Recency'] = (pd.Timestamp('20151201') - recent_purchase_days['de_dt']).dt.days
```

② 고객별 소분류명별 구매 빈도

```
purchase_frequency = filter_period.groupby(['cno','gender', 'age_grp', 'scls_c_nm'])['rct_seq'].nunique().reset_index(name='Frequency')
```

③ 총 구매금액 구하기

```
purchase_monetary = filter_period.groupby(['cno','gender', 'age_grp', 'scls_c_nm'])['buy_am'].sum().reset_index(name='Monetary')
```

→ 3가지 컬럼을 고객번호 상품명 기준으로 merge 하여 최종 데이터셋 선정.

RFM별 score 산출

- 세 가지 변수(최근 구매일, 구매 빈도 및 구매 금액) 각각에 점수를 할당.
- 최근 구매일 점수는 최근성 열의 값을 가장 높은 것부터 낮은 것까지 순위를 매겨 계산.
- 구매 빈도 점수는 빈도 열, 금액 점수는 금액 열의 값을 가장 낮은 값에서 가장 높은 값으로 순위를 매겨 계산.

rank 함수 이용

```
df_03_RFM_quan['Recency_Score'] = df_03_RFM_quan['Recency'].rank(method='min', ascending=False)
df_03_RFM_quan['Frequency_Score'] = df_03_RFM_quan['Frequency'].rank(method='min', ascending=True)
df_03_RFM_quan['Monetary_Score'] = df_03_RFM_quan['Monetary'].rank(method='min', ascending=True)
```

+) RFM 범위 컬럼 생성

- Recency 값이 0(Recency_start) ~ 236(Recency) 사이에 있다면 Recency_Score를 10으로 봄.
- 한 행 위로 이동한 다음 누락된 값을 0으로 채움.

shift 함수 이용

```
df_03_RFM_quan['Recency_start'] = (df_03_RFM_quan['Recency'].shift(1)+1).fillna(0)
df_03_RFM_quan['Frequency_start'] = (df_03_RFM_quan['Frequency'].shift(1)).fillna(0)
df_03_RFM_quan['Monetary_start'] = (df_03_RFM_quan['Monetary'].shift(1)+1).fillna(0)
```


RFM Score 생성

- 각각의 Score에 임의의 가중치를 곱하여 최종 점수 생성
- 추천 시스템 이용시 필요 항목

$$(\text{Recency score} \times \text{Recency weight}) + (\text{Frequency score} \times \text{Frequency weight}) + (\text{Monetary score} \times \text{Monetary weight})$$

→ Recency_Score 열에 0.4, Frequency_Score 열에 0.3, Monetary_Score 열에 0.3을 곱한 다음 모든 값을 더하여 계산

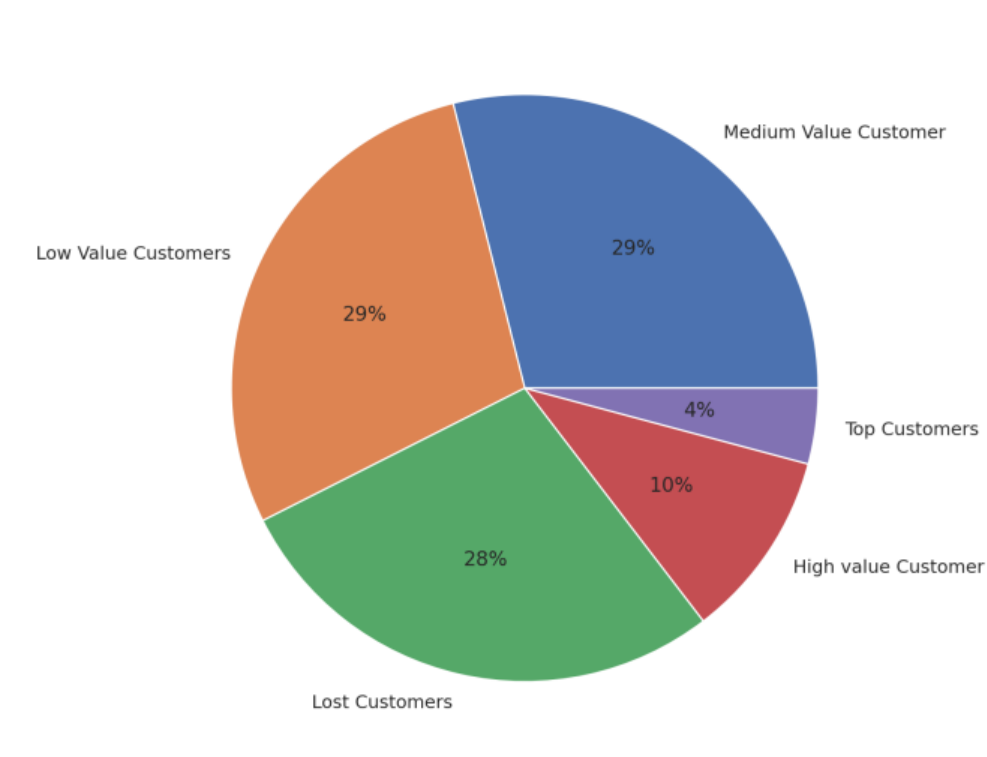


Formula used for calculating rfm score is :

$$0.15 \times \text{Recency score} + 0.28 \times \text{Frequency score} + 0.57 \times \text{Monetary score}$$

Rating Customer based upon the RFM score

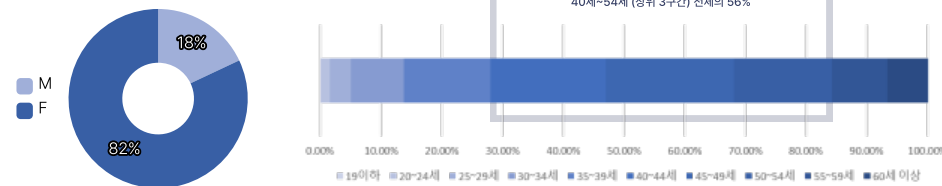
- rfm score > 4.5 : Top Customer
- $4.5 > \text{rfm score} > 4$: High Value Customer
- $4 > \text{rfm score} > 3$: Medium value customer
- $3 > \text{rfm score} > 1.6$: Low-value customer
- rfm score < 1.6 : Lost Customer



군집화 1) Kmeans

군집분석으로 고객 특성에 맞게 군집 생성

성별과 나이만 이용



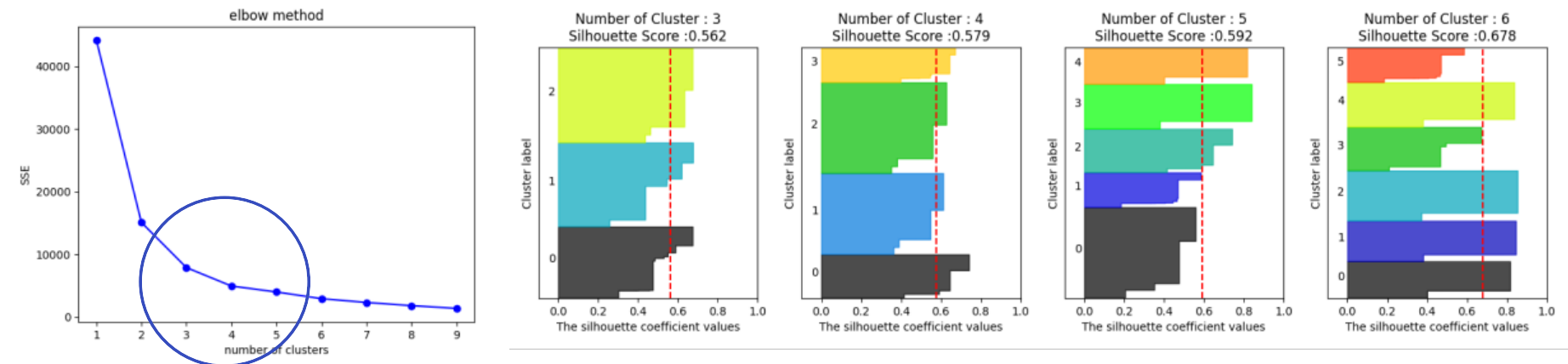
- 여성고객이 82%
- 연령대 45~49세가 21%, 남,여 연령대 비율은 유사

다른 컬럼 추가

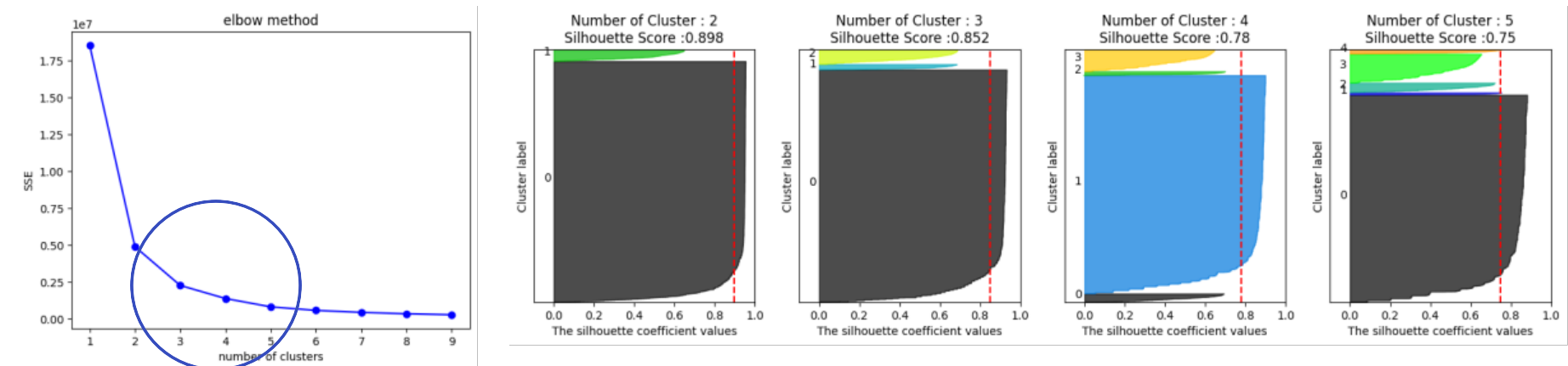
- Recency, Frequency, Monetary 추가해가면서 군집 분석
- 가장 좋은 형태를 보인 것은 Frequency 추가했는 경우

(전처리)

- 성별, 연령대 LabelEncoder() 이용
- MinMaxScaler()로 RFM 정규화



→ 엘보우 기법으로 적절해 보이는 군집 개수를 찾았지만, 실루엣 계수가 낮은것을 보아 다른 군집과 가까울 것으로 볼 수 있음.



→ 0.75~0.898로 실루엣 계수가 1에 가깝지만 군집이 고루게 되지 않은것을 그래프로 확인할 수 있음.

군집화 1) Kmeans

군집분석으로 고객 특성에 맞게 군집 생성

변수 추가

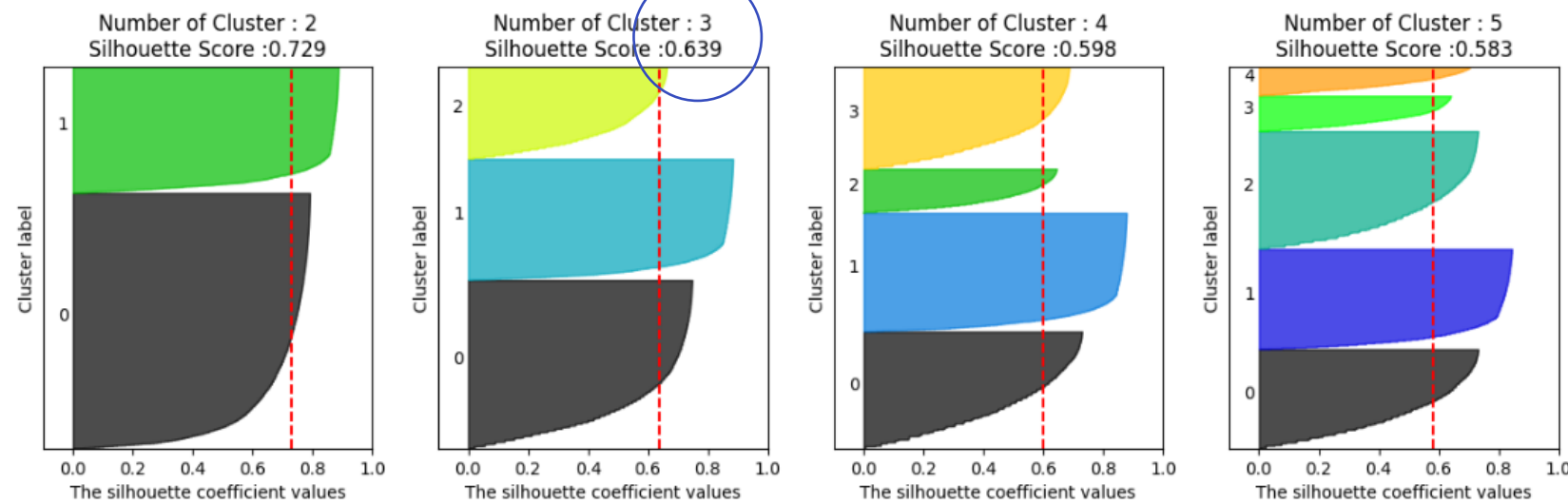
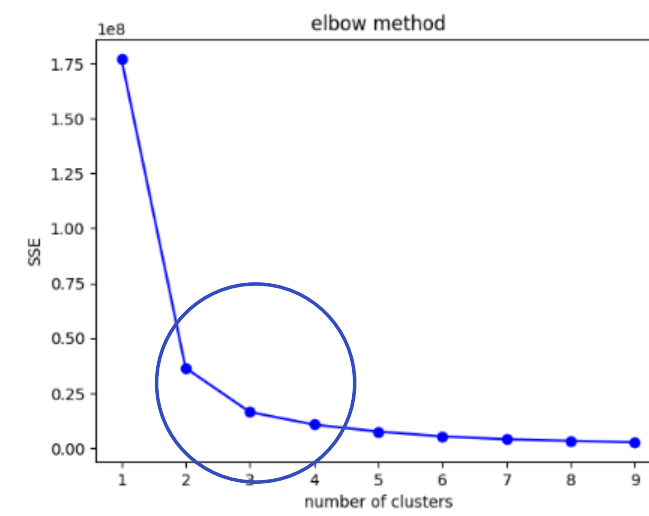
- 성별과 연령대에 Frequency 컬럼을 추가했을 경우



(최종 군집1)

→ 나이, 성별, Frequency 이용하여

3개의 군집 분류

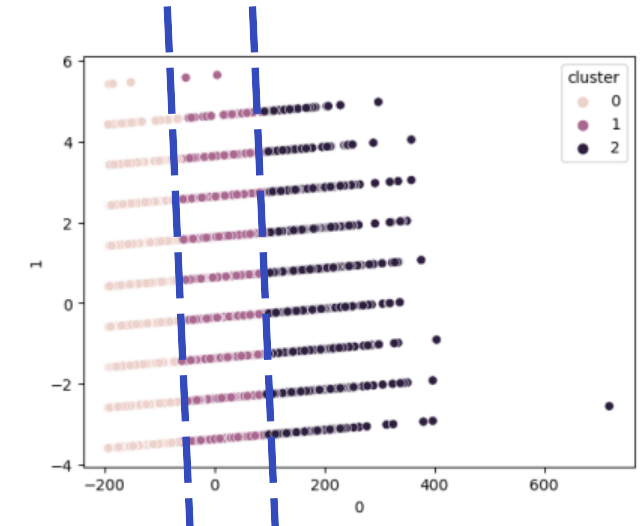


→ 0.729 (군집수2), 0.639(군집수3) 으로 비교적 실루엣 계수가 높아지고 군집이 된 것으로 보임.

군집 시각화

- PCA를 이용해 4개의 속성을 2개로 차원 축소한 후 시각화.

```
from sklearn.decomposition import PCA
# 객체
pca = PCA(n_components=2)
# 적용
pca.fit(data_pca)
x_pca = pca.transform(data_pca)
# 보기 쉽게 데이터프레임으로
pca_df = pd.DataFrame(x_pca)
pca_df['cluster'] = X_final['cluster']
# 그래프 그리기
sns.scatterplot(x=2, y=1, hue='cluster', data=pca_df)
```

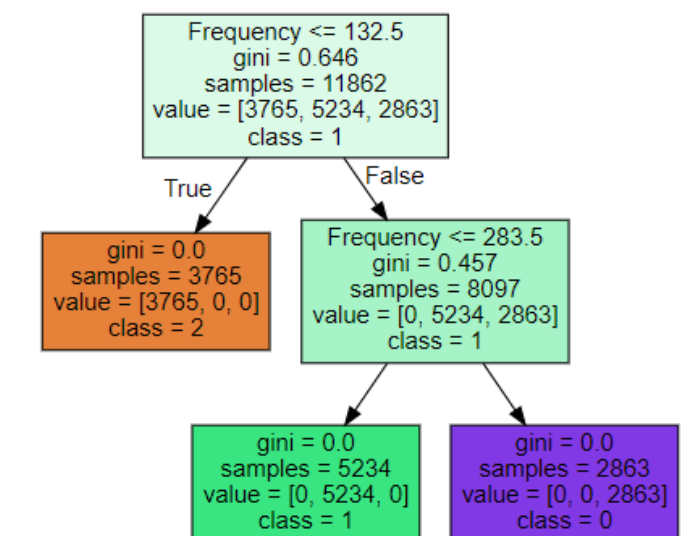


Tree 그래프

```
from sklearn.tree
import DecisionTreeClassifier
```

→ 분류에 가장 주요하게 쓰인 변수는 'Frequency' 인것을 알 수 있음.

군집	고객수	구매 빈도
군집1	2863	F <= 132
군집2	5234	F <= 283
군집3	3765	F > 283

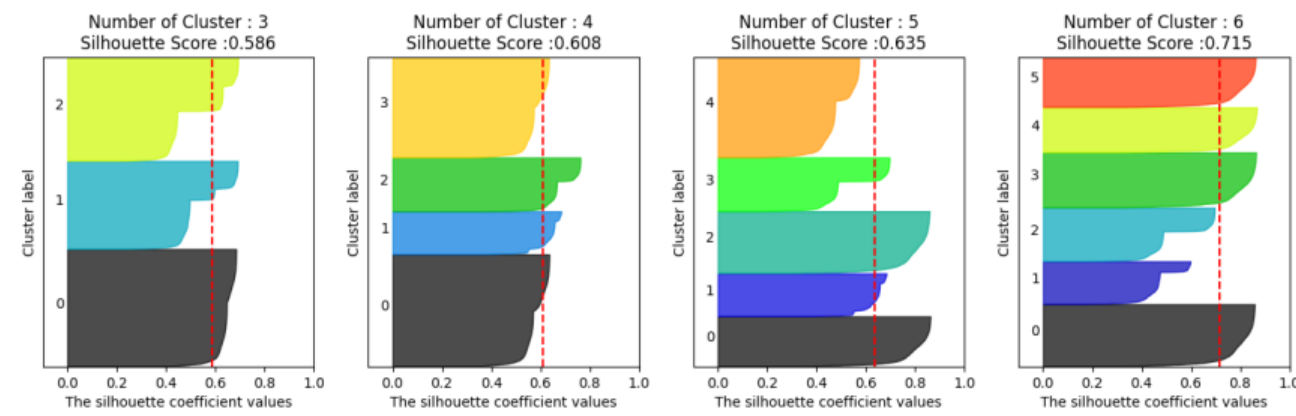


군집화 1) Kmeans

군집분석으로 고객 특성에 맞게 군집 생성

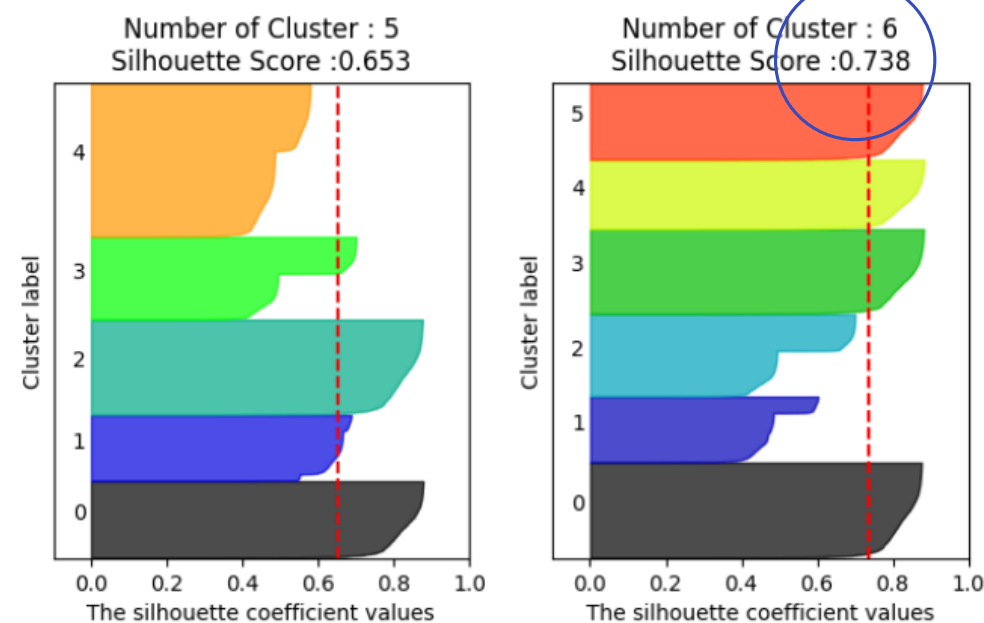
성별 제외

- 성별을 제외하고 군집 확인 시 더 분류가 잘되어 보임.



(최종 군집2)

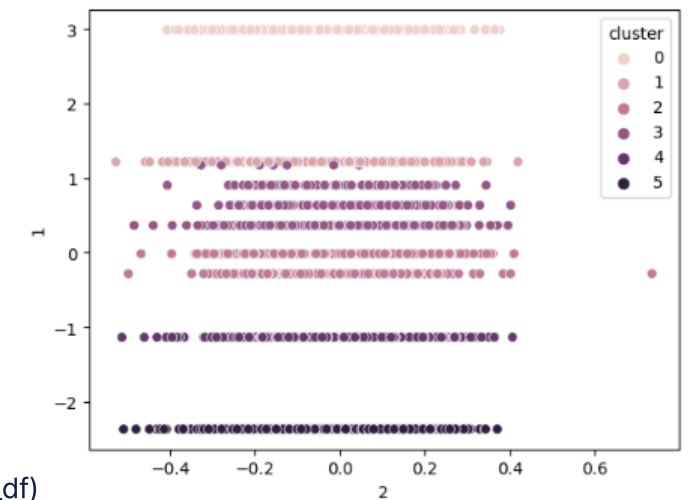
→ 나이, Frequency,
Monetary, Purchase_cycle
이용하여 6개의 군집 분류



군집 시각화

- PCA를 이용해 4개의 속성을 2개로 차원 축소한 후 시각화.

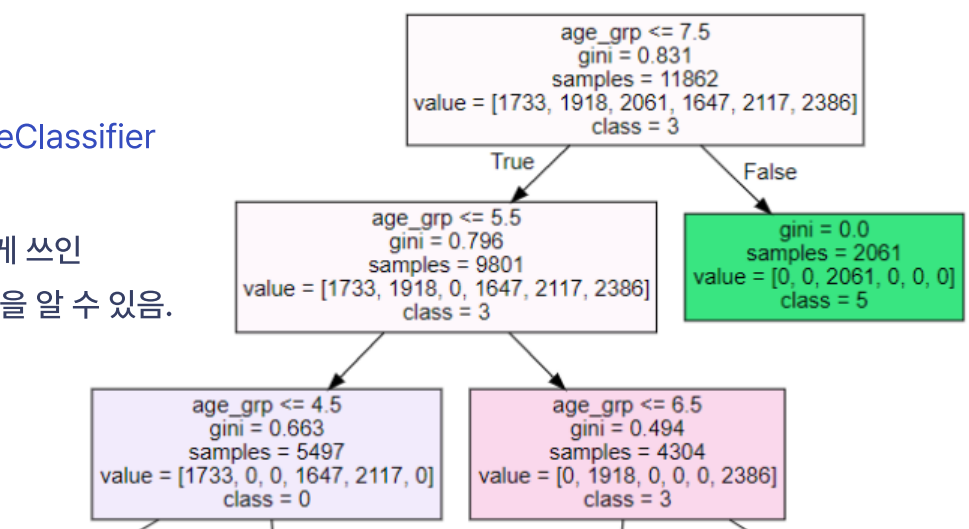
```
from sklearn.decomposition import PCA
# 객체
pca = PCA(n_components=3)
# 적용
pca.fit(data_pca)
x_pca = pca.transform(data_pca)
# 보기 쉽게 데이터프레임으로
pca_df = pd.DataFrame(x_pca)
pca_df['cluster'] = X_final['cluster']
# 그래프 그리기
sns.scatterplot(x=2, y=1, hue='cluster', data=pca_df)
```



- Tree 그래프

```
from sklearn.tree
import DecisionTreeClassifier
```

→ 분류에 가장 주요하게 쓰인
변수는 '연령대' 인것을 알 수 있음.



군집화 2) DBSCAN

군집분석으로 고객 특성에 맞게 군집 생성

| Kmeans와 비교

(최종 군집2)

→ 나이, Frequency, Monetary, Purchase_cycle

이용하여 6개의 군집 분류 이용하여 DBSCAN 분석과 비교

```
from sklearn.cluster import DBSCAN
```

R와 역할, 반지름 지정

epsilon = 0.3

minimumSamples 최소 요소 개수

```
db = DBSCAN(eps=epsilon, min_samples=4).fit(X)
```

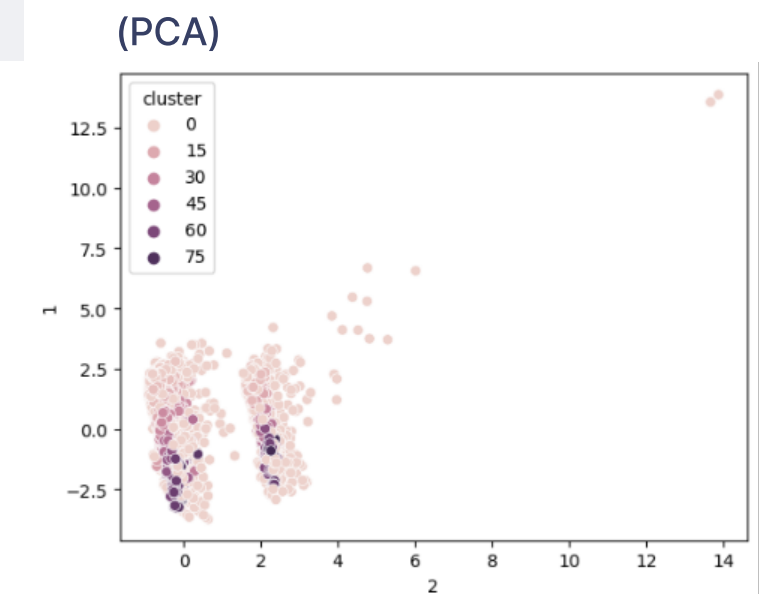
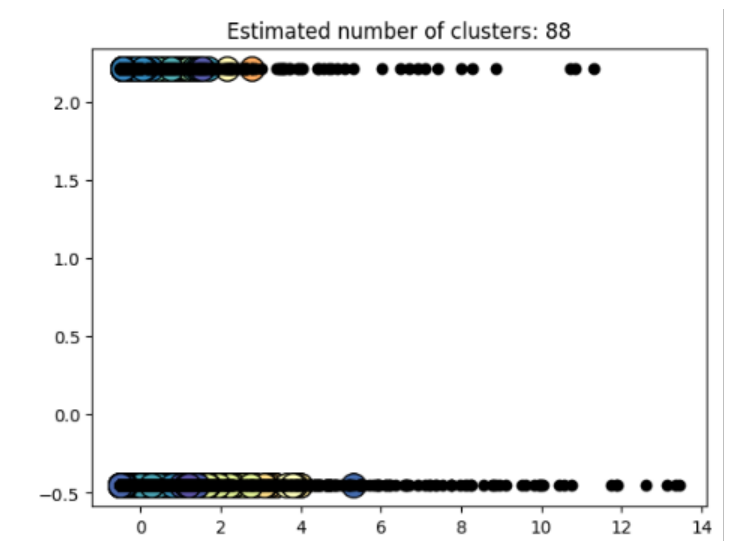
```
labels = db.labels_
```

레이블의 클러스터 수

```
n_clusters_ = len(set(labels)) - (1 if -1 in labels else 0)
```

- Estimated number of clusters: 88
- Estimated number of noise points: 1583
- Homogeneity: 0.867
- Completeness: 0.541
- V-measure: 0.666
- Adjusted Rand Index: 0.504
- Adjusted Mutual Information: 0.663
- Silhouette Coefficient: -0.02

→ 연령대로 주로 나뉜 Kmeans 군집 분석에 이용한 데이터를 DBSCAN에 넣었을때 Kmeans 군집 결과와 DBSCAN결과와 비교함. (동질성 : 0.867, 완전성 0.541)
→ 평가가 너무 낮은건 아니지만 그래프로 그려봤을 때 군집이 잘 분류되지 않음을 볼 수 있음.



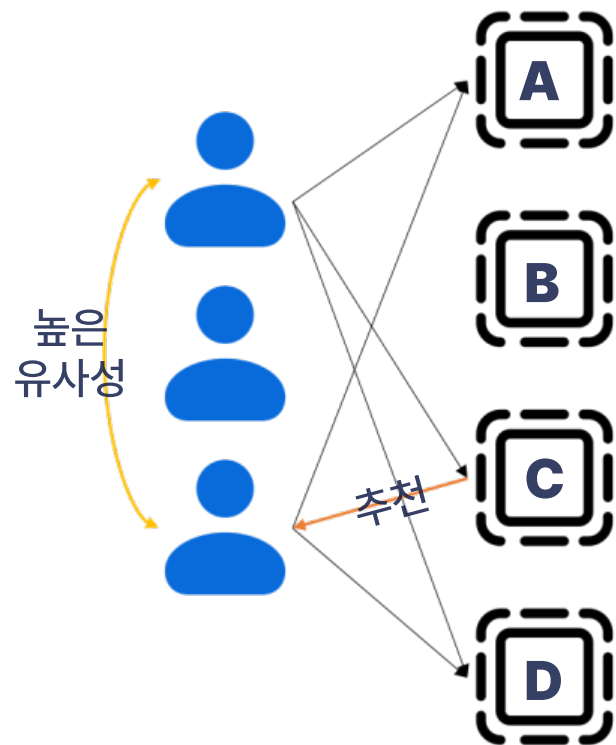
협업 필터링(CF)

- 사용자의 취향을 고려한 추천시스템 개발
- 특정 아이템에 대해 비슷한 취향을 가진 사람들은 다른 아이템 또한 비슷한 취향을 가질 것이라고 가정
- 특정 사용자와 취향이 비슷한 집단 (유사 집단) 의 취향을 기반으로 아이템 추천

사용자 기반 협업 필터링

(사용자(고객) 간의 유사성을 측정)

User CF - row : 사용자 , column : 아이템, values : 평가점수



Pivot 만들기 + 표준화

```
piv = merged.pivot_table(index=['cno'], columns=['scls_c_nm'], values='rmf_score')
piv_norm = piv.apply(lambda x: (x-np.mean(x))/(np.max(x)-np.min(x)), axis=1) # min-max scaling
piv_norm.fillna(0, inplace=True)
```

(17953, 594)

[illegible]

유사도 계산

사용자 또는 아이템간 유사도를 구하기 위해 유사도 지표를 활용

* 유사도 계산, **scipy**의 **csr_matrix**

```
piv_sparse = sp.sparse.csr_matrix(piv_norm.values)
```

Compressed Sparse Row (CSR) Matrix

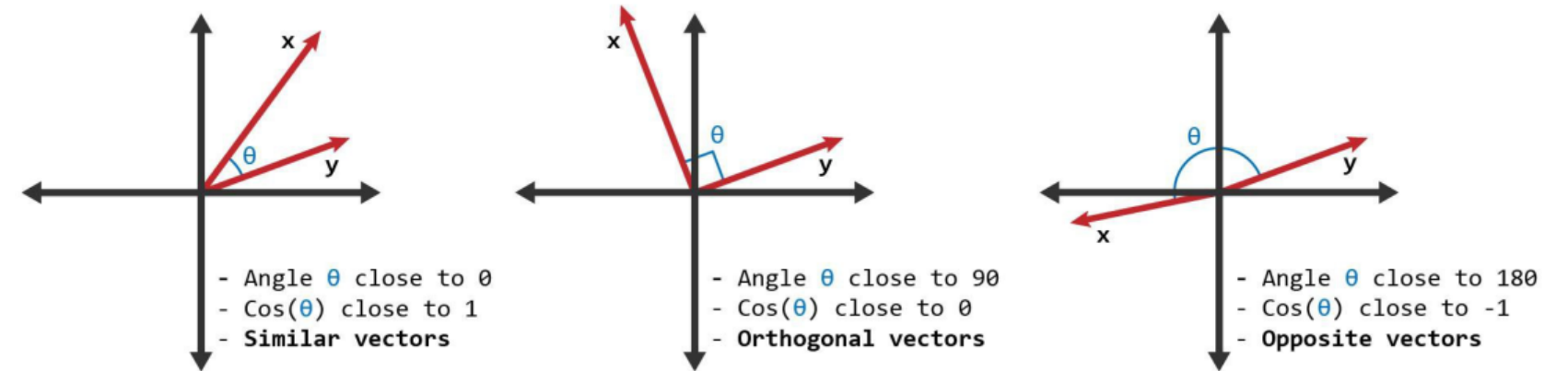
- 행렬의 값이 대부분 '0'인 행렬인 **희소행렬**
- 대부분의 값이 '0'이므로 이를 그대로 사용할 경우 메모리 낭비가 심하고 또 연산시간도 오래 걸리는 단점
- 0을 제외한 데이터를 저장
- 0이 많다는 것은 sparse하다는 것, 따라서 0을 제외한 데이터를 사용해야 연산을 빨리할 수 있음.

$$A = \begin{bmatrix} 1 & 0 & 2 & 0 \\ 0 & 0 & 0 & 0 \\ 1 & 0 & 2 & 3 \\ 0 & 1 & 0 & 2 \end{bmatrix} \quad \begin{matrix} \text{row_ptr}[] = [0 & 2 & 2 & 5 & 7] \\ \text{col_idx}[] = [0 & 2 & 0 & 2 & 3 & 1 & 3] \\ \text{val}[] = [1 & 2 & 1 & 2 & 3 & 1 & 2] \end{matrix}$$

→ A sparse matrix and its CSR format.

(압축희소행렬 만든 후
코사인 유사도 함수적용)

Cosine_similarity



cosine_similarity 함수 이용

```
from sklearn.metrics.pairwise import cosine_similarity
user_similarity = cosine_similarity(piv_sparse)
```

→ 사용자 유사성 측정

+) 아이템 기반

(고객 선호상품과 다른 상품들간의 유사성을 측정)

```
item_similarity = cosine_similarity(piv_sparse.T)
```

→ row : 아이템, column : 사용자, values : 평가점수

유사도 지표 이용한 상품 추천

사용자 기반 협업 필터링

(사용자(고객) 간의 유사성을 측정)

- top_users(214) << 유사 사용자 top 10, 확인



Most Similar Users:

User #3011, Similarity value: 0.46	User #18972, Similarity value: 0.40
User #9390, Similarity value: 0.42	User #878, Similarity value: 0.40
User #4575, Similarity value: 0.41	User #3797, Similarity value: 0.40
User #2320, Similarity value: 0.41	User #636, Similarity value: 0.40
User #238, Similarity value: 0.41	User #14171, Similarity value: 0.39

아이템 기반 협업 필터링

(고객 선호상품과 다른 상품들간의 유사성을 측정)

- top_item('화과자') << 유사 상품 top 10, 확인



Similar shows to 화과자 include:

No. 1: 특산물행사(0.17)	No. 6: 떡(0.15)
No. 2: 햄(0.17)	No. 7: 김류(0.14)
No. 3: 일용잡화(0.17)	No. 8: 아이스크림(0.14)
No. 4: 중식델리(0.16)	No. 9: 어묵(0.13)
No. 5: 냉동식품(0.15)	No. 10: 멸치류(0.13)

(top 10)

- 사용자, 아이템 기반 추천 top 10

- This function will return the top 10 users with the highest similarity value

```
def top_users(user):  
    if user not in piv_norm.columns:  
        return('No data available on user {}'.format(user))  
  
def top_item(item_nm):  
    count = 1  
    print('Similar shows to {} include:\n'.format(item_nm))  
    result = item_sim_df.loc[~item_sim_df.index.isin([item_nm]), item_nm]  
        .sort_values(ascending = False)[:10]  
    for item, score in result.items():  
        print('No. {}: {}({:.2f})'.format(count, item , score))  
        count +=1
```


행렬분해(MF)의 SVD : 특이값 분해

- SVD(Singular value Decomposition)
- 특이값 분해 모델 (SVD) 을 통해서 추천시스템 개발

$$R (4 \times 5) = P (4 \times 2) \times Q.T (2 \times 5)$$

	item 1	item 2	item 3	item 4	item 5
User1	4			2	
User2		5		3	
User3			3	4	4
User4					
User5	5	2	1	2	

	Factor 1	Factor 2
User1	0.94	0.96
User2	2.14	0.08
User3	1.93	1.79
User4	0.58	1.59

	item 1	item 2	item 3	item 4	item 5
Factor1	1.7	2.3	1.41	1.36	0.41
Factor2	2.49	0.41	0.14	0.75	1.77

Matrix Factorization-based algorithms

```
from surprise import SVD
# Use the famous SVD algorithm.
algo = SVD()
```

- Surprise 외부의 데이터를 사용해야 하는 경우, DataFrame 형태로 읽고 Reader 클래스에 평가 척도를 지정한 후 Surprise의 Dataset 클래스로 읽어오면 됨.

1) surprise 패키지 사용

```
from surprise import Dataset
from surprise import Reader
from surprise.model_selection import train_test_split

reader = Reader(rating_scale=(df['RFM_SCORE'].min(), df['RFM_SCORE'].max()))
data = Dataset.load_from_df(df[['Customer', 'Product', 'RFM_SCORE']], reader)

→ surprise 패키지는 유저/아이템/평점 순서대로 데이터를 순서대로 넘겨주어야 함.
```

2) GridSearchCV 파라미터 튜닝

```
from surprise.model_selection import GridSearchCV

param_grid = {'n_factors': [50, 100], 'lr_all': [0.02, 0.01],
              'reg_all': [0.02, 0.01], 'n_epochs': [40, 50]}
gs = GridSearchCV(algo_class = SVD, measures=['RMSE'],
                  param_grid=param_grid)
gs.fit(data) # 학습

→ 하이퍼파라미터를 변경해가며 rmse 값을 최저치로 낮추는 튜닝 수행.
```

3) 평가 (RMSE)

```
from surprise import accuracy

prediction = algo.test(test_data)
RMSE = accuracy.rmse(prediction)

→ 최적의 파라미터로 학습 및 예측한 데이터 평가(accuracy)
```

top-N 정확도 계산

- 사용자별로 10개의 추천 아이템을 생성
- predictions의 각 레코드는 (사용자 ID, 아이템 ID, 실제 점수, 예측 점수)으로 구성

1) top 10개

```
for uid, iid, true_r, est, _ in predictions:
    # 사용자별로 아이템과 예측 점수를 같이 저장
    top_n[uid].append((iid, est))

# top_n을 사용자ID를 가지고 예측 점수가 높은 n개의 아이템으로 교체
for uid, user_ratings in top_n.items():
    user_ratings.sort(key=lambda x: x[1], reverse=True)
    top_n[uid] = user_ratings[:n]
```

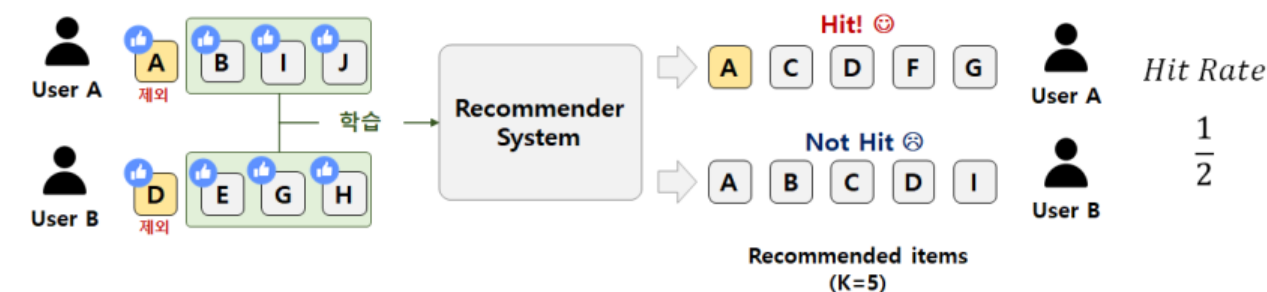
→ 예측 점수는 SVD의 test의 리턴값

2) 데이터 셋

```
from surprise.model_selection import LeaveOneOut
LOOCV = LeaveOneOut(n_splits=1, random_state=1)
```

→ 각 사용자별로 하나의 점수만을 테스트셋에 보관 (n_splits가 1 즉 1개의 폴드)

• Hit Rate@K



3) 학습 및 평가

🔍 result ⇒ 24.761

top N 정확도를 계산

hits = 0

total = 0

ratingCutoff = 4.0

남겨졌던 데이터의 사용자들을 대상으로

cutoffRating이상의 평점을 갖는 아이템이 추천되었는지 확인

for userID, leftOutitemID, rating in testSet :

높은 평점이 매칭되는 경우만 고려

if (rating >= ratingCutoff):

사용자의 추천 리스트를 확인

for itemID, rating in topNPredicted[userID]:

해당 사용자의 추천 아이템에 해당하는 경우에만 카운트

if (leftOutitemID == itemID):

hits += 1

break

total += 1

print(hits/float(total)*100.)

추천알고리즘 간단구현

1) 데이터

- 필요한 데이터만 저장해서 이용

```
# 데이터 저장
cus.to_pickle('./data/cus.pkl') < 고객 군집 정보
score.to_pickle('./data/item_score.pkl')
< rmf_score 정보
```

2) 함수이용

- 함수 형태로 작성하여 코드를 모듈화

```
# 함수로 쪼개기
def data_read(num, cus, rating): < 데이터 load, 군집쪼개기
    ... return ...
def cal_similarity(df): < 유사도계산
    ... return ...
```

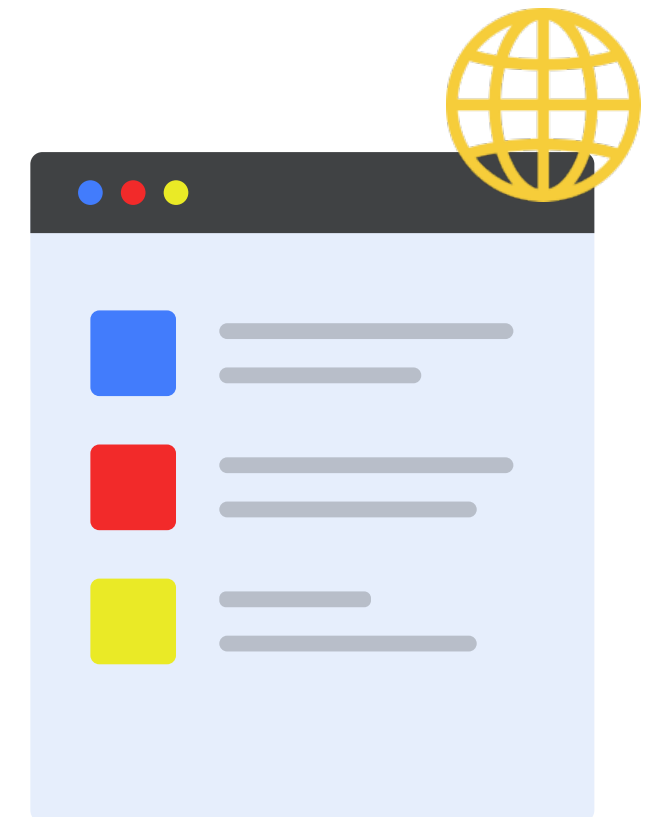
3) Flask 활용

- 파이썬으로 작성된 마이크로 웹 프레임워크
- 파일 하나로 구성된 짧은 코드만으로도 완벽하게 동작하는 웹 프로그램을 만들 수 있음.

```
from flask import Flask, render_template
# 플라스크 애플리케이션을 생성
app = Flask(__name__)

@app.route('/')
def main():
    return render_template('index.html')
    < html 렌더링해서 보여주도록

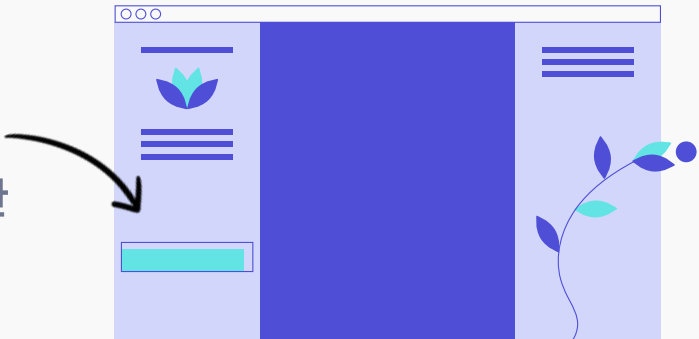
# flask 실행
> flask run
```



추천알고리즘 간단구현

활용 예시

- 배너를 통한 추천상품 노출
 - 구매율 높은 상품
 - 사용자와 구매패턴이 유사한 다른 사용자가 구매한 상품



- 고객맞춤 뉴스레터 발송



- 구매빈도 높은상품 할인쿠폰 발송
- 구매관련상품 신제품 알림
- 기념일 축하쿠폰
- 할인 행사 정보



Recommended items for the user:

Cluster ID: 3

Item	Distance
global SPA	0.05236687889755947
식당가 양식	0.052717494357407135
색조 화장품	0.052918571867652996
유기농채소	0.05518229450990032
이지캐주얼	0.05549612748005805
내의	0.05584181168746905
영스트리트	0.05588325878287337
수영복	0.0564576591816388
농산가공	0.05737278321746869
스타킹(특정)	0.05812611988281581

[Go back](#)



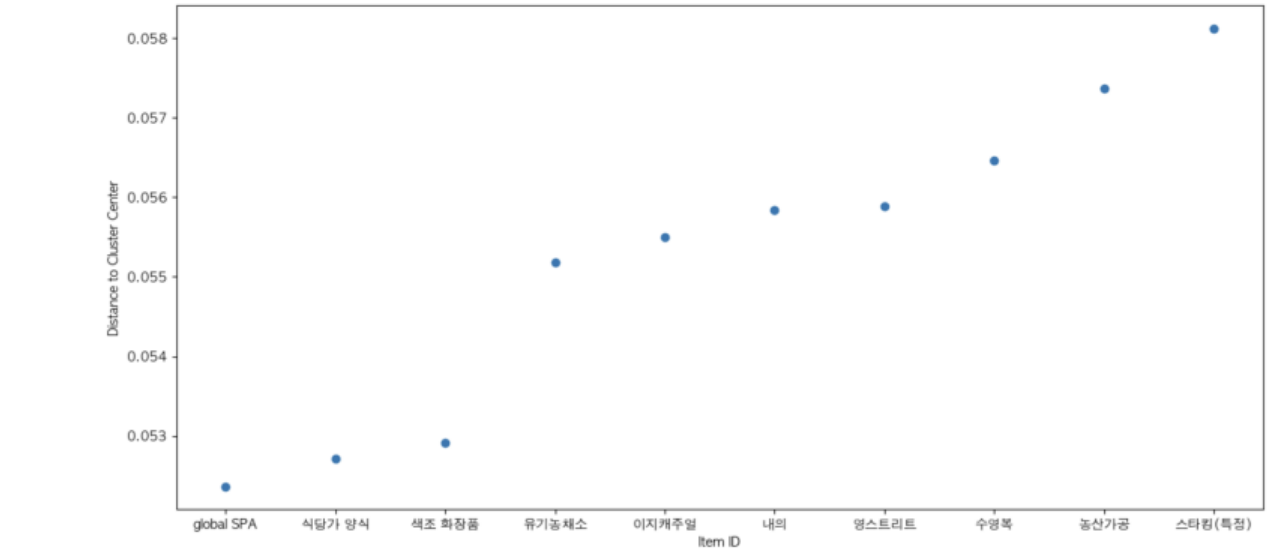
- Flask 구현



Recommended items for the user:

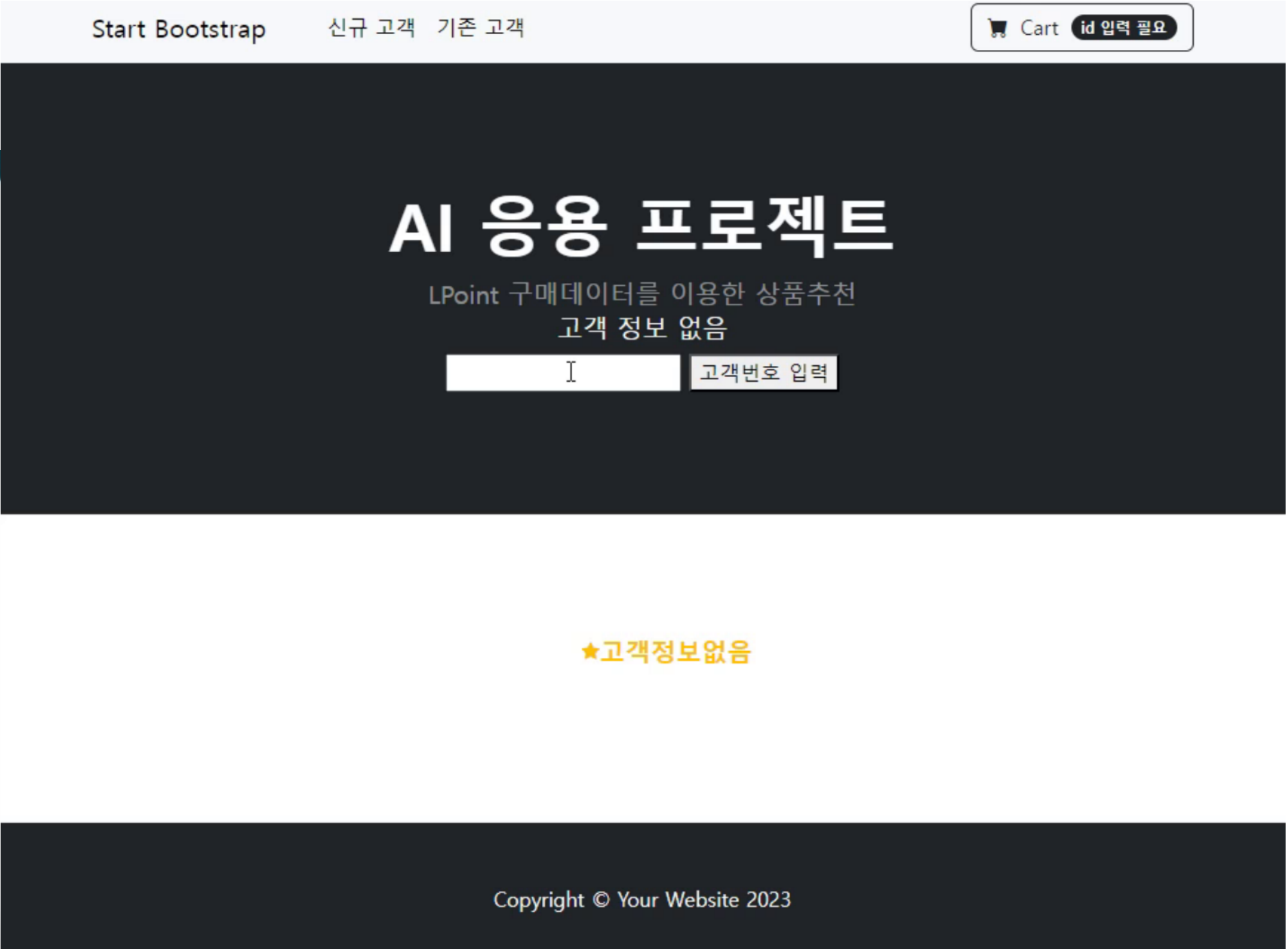
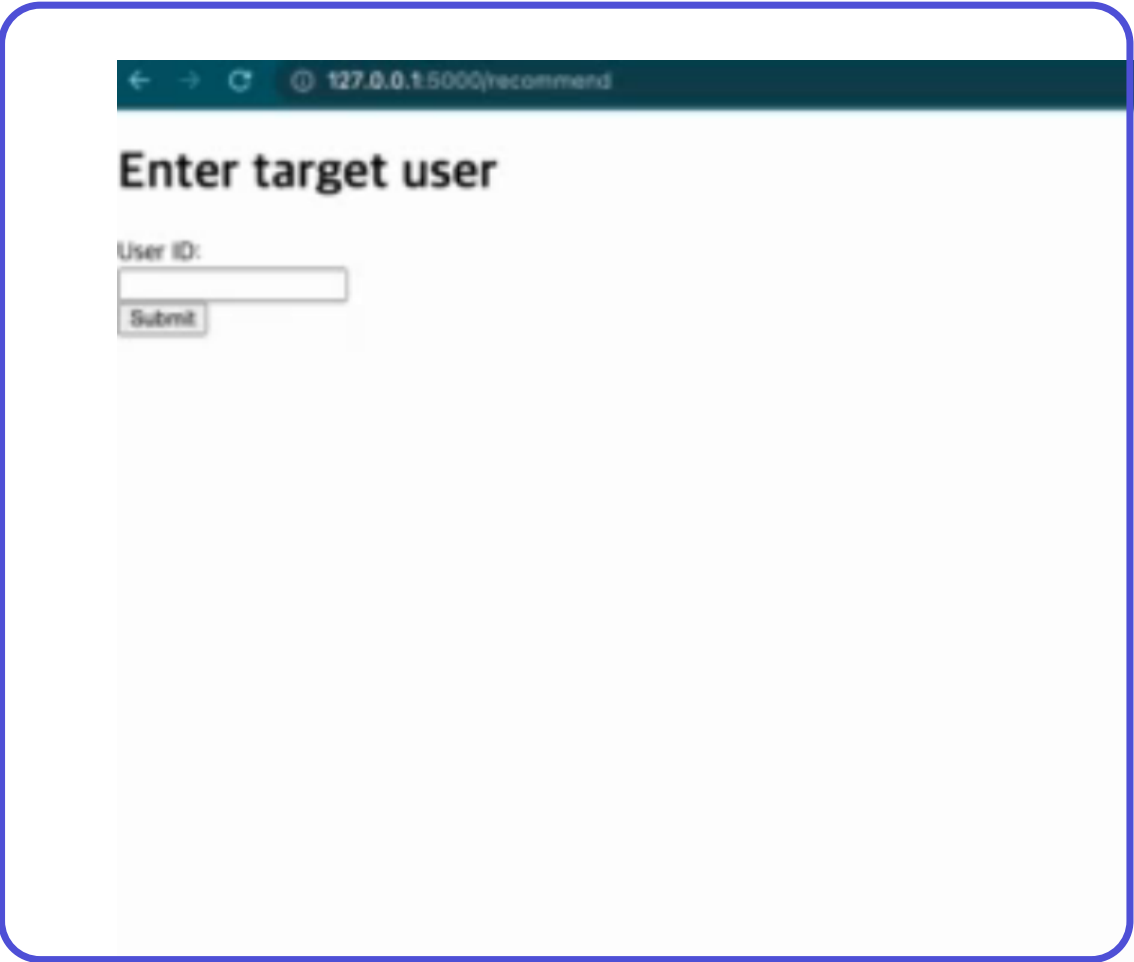
Cluster ID: 3

Item	Distance
global SPA	0.05236687889755947
식당가 양식	0.052717494357407135
색조 화장품	0.052918571867652996
유기농채소	0.05518229450990032
이지캐주얼	0.05549612748005805
내의	0.05584181168746905
영스트리트	0.05588325878287337
수영복	0.0564576591816388
농산가공	0.05737278321746869
스타킹(특정)	0.05812611988281581



[Go back](#)

프로젝트 결과



Thank you

✉ isdl15980@gmail.com
jinkyu.lee@gmail.com
qwe121292@gmail.com



국민대학교 소프트웨어융합대학원
AI 응용시스템의 이해와 구축

